

1635

The Website Handbook

NEWTOWN TECHNOTICS - FRC TEAM 1635

EVERY FILE - EVERY SYSTEM - EVERY ROUTINE JOB

LOGBOOK POSTING - 3D ROBOT - SCROLL ANIMATION

noirhit.github.io/1635-technotics-site

Table of Contents

1. What this website is	3
2. The tools you need	4
3. Running the site on your computer	5
4. Project map — every file explained	6
5. Anatomy of a page	8
6. The design system (css/style.css)	10
7. The JavaScript files	12
8. TUTORIAL: Posting to the Logbook	13
9. TUTORIAL: The 3D robot, from zero	15
10. TUTORIAL: The scroll animations, from zero	18
11. The model build pipeline (tools/build_kitbot.py)	20
12. The contact form	21
13. Publishing to the live site	22
14. Cookbook — common jobs in 5 minutes	23
15. Troubleshooting	24
16. Glossary	25

The complete owner's manual for [1635-technotics-site](#).

This document explains every file in the repository, how each system works, and how to do every routine job — posting to the Logbook, updating the 3D robot, changing colors, publishing to the live site. It is written for a beginner: if you can edit a text file and click buttons in GitHub Desktop, you can run this website. If you read the whole thing, you could rebuild the site from a blank folder.

1. What this website is

The site is **plain HTML + CSS + JavaScript**. There is no framework (no React, no Vue), no build step, no `npm install`. What you see in the folder is exactly what the browser receives. That is a deliberate choice: anyone on the team can open a file, change a line, and understand what happened.

Three external libraries load from CDNs (public file servers) at runtime:

Library	What it does for us	Loaded on
three.js (r128)	Renders the 3D robot in the browser (WebGL)	index, robot
GLTFLoader	The three.js add-on that reads our <code>.glb</code> model file	index, robot
GSAP + ScrollTrigger	Scroll-linked animations (parallax, staggered grids)	all pages

Everything else — the reveal animations, the slideshow, the blog, the nav — is hand-written vanilla JavaScript in the `js/` folder.

Hosting: GitHub Pages serves the `main` branch of the `Noirhit/1635-technotics-site` repository at <https://noirhit.github.io/1635-technotics-site/>. Every push to `main` redeploys the site automatically in about a minute.

Design direction (so future edits stay coherent): the site is styled as a *machine-shop drawing package* — dark graphite surfaces, warm chalk-white text, JetBrains Mono "dimension callout" labels, and exactly one accent hue (red) used sparingly. Big display type is Clash Display. If you add a new section, reuse the existing classes and it will automatically match.

2. The tools you need

Tool	Why	Where
VS Code	Editing files	code.visualstudio.com
Live Server (VS Code extension)	Runs a local web server — required for the blog and 3D model to work locally	VS Code -> Extensions -> search "Live Server" -> Install
GitHub Desktop	Sending your changes to the live site without command-line git	desktop.github.com
Python 3 + numpy (optional)	Only needed to rebuild the 3D model from CAD	python.org , then <code>pip install numpy</code>

You do **not** need Node.js, npm, or any package manager.

3. Running the site on your computer

Why you can't just double-click `index.html`: two features use `fetch()` — the Logbook (loads `data/posts.json`) and the robot (loads `models/kitbot.glb`). Browsers block `fetch()` from `file://` pages for security. A local server fixes this.

1. Open the project folder in VS Code (`File -> Open Folder`).
2. Right-click `index.html` in the file tree -> **Open with Live Server**.
3. Your browser opens `http://127.0.0.1:5500/index.html`. Every file save auto-reloads the page.

That's the entire dev loop: edit -> save -> look.

4. Project map — every file explained

```
1635-technotics-site/
■■■ index.html           Home page
■■■ about.html          The Team page
■■■ robot.html          The Robot page (3D exploded view lives here)
■■■ blog.html           Logbook page
■■■ sponsors.html       Backers page
■■■ contact.html        Contact / join page (working form)
■
■■■ css/
■   ■■■ style.css       ALL styling for every page (~1300 lines)
■
■■■ js/
■   ■■■ site.js         Functional code: nav, blog, modal, slideshow, form
■   ■■■ motion.js       Animation code: reveals, counters, nav-hide, GSAP
■   ■■■ rig.js          3D code: loads the robot, hero orbit, exploded scroll
■
■■■ data/
■   ■■■ posts.json      THE BLOG DATABASE – edit this to post (see §8)
■
■■■ models/
■   ■■■ kitbot.glb      The 3D robot (3.5 MB binary, built by tools/)
■
■■■ tools/
■   ■■■ build_kitbot.py Converts the raw CAD export into kitbot.glb (see §11)
■
■■■ images/             All photos (team, robot, sponsors, blog headers)
■■■ assets/
■   ■■■ kitbot/         Raw CAD export (941 MB, 621 files) – gitignored,
                        never uploaded; only the input to build_kitbot.py
■
■■■ .gitignore          Tells git to ignore assets/kitbot/ and *.CR2 files
■■■ .nojekyll           Tells GitHub Pages to skip its "Jekyll" processor
■■■ HANDBOOK.md         This document
```

The six HTML pages

Each page is self-contained and shares the same skeleton (see §5). Quick inventory of what is unique on each:

Page	Unique sections	Special JS
<code>index.html</code>	Hero with 3D robot + big type, ticker, mission band, about/robot/logbook/backers previews, "on the record" grid	<code>rig.js</code> hero mount, blog "latest"
<code>about.html</code>	Team photo (<code>.crew</code>), mission split, subteam grid, horizontal build-season rail, roster lists	rail nudge
<code>robot.html</code>	Spec board, pinned exploded-view stage , three subsystem cards with image reels, season archive	<code>rig.js</code> exploded mount
<code>blog.html</code>	Post grid (<code>#feed</code>) — rendered entirely by JS from <code>posts.json</code>	blog renderer + modal
<code>sponsors.html</code>	Tiered backer cards, "where the money goes" sheet	—

Page	Unique sections	Special JS
<code>contact.html</code>	Join checklist, contact rail, working Web3Forms form	form submit handler

Files you should never need to touch

- `.nojekyll` — an empty file. Without it, GitHub Pages runs a site generator called Jekyll over the repo, which is slow and can mangle files.
- `.gitignore` — keeps the 941 MB CAD dump and raw camera files out of git.

If you add other huge files you don't want uploaded, list them here.

- `models/kitbot.glb` — generated. Don't hand-edit; re-run the build tool.

5. Anatomy of a page

Open any page and you'll find the same skeleton. Understanding it once means understanding all six pages.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page – 1635 Newtown Technotics</title>

  <!-- fonts: Clash Display + Satoshi from Fontshare, JetBrains Mono from Google -->
  <link href="https://api.fontshare.com/v2/css?f[]=clash-display@500,600,700&f[]=satoshi@400,500,700&display=swap" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500;600&display=swap" rel="stylesheet">

  <!-- the "js flag": adds class="js" to <html> BEFORE css loads.
       All hide-then-animate CSS is gated on `js` – so if JavaScript is
       off or broken, content is simply visible instead of hidden forever -->
  <script>document.documentElement.className+=" js";</script>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <header class="nav"> ... logo, links, burger button, progress hairline ... </header>

  <main>
    <section class="bay">          <!-- one horizontal band of content -->
      <div class="shell">          <!-- centred, max-width 1440px column -->
        <div class="titleblock">  <!-- section heading group -->
          <span class="note note--red titleblock__tag">$ 01 – Label</span>
          <h2>Heading.</h2>
          <p>Optional standfirst.</p>
        </div>
        ... content ...
      </div>
    </section>
    ... more bays ...
  </main>

  <footer class="foot"> ... big 1635, link columns, status line ... </footer>

  <!-- scripts load LAST so the page paints before JS runs -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/ScrollTrigger.min.js"></script>
  <script src="js/motion.js"></script>
  <script src="js/site.js"></script>
</body>
</html>
```

Key layout vocabulary (all defined in `style.css`):

Class	Meaning
<code>.bay</code>	A full-width horizontal section band with vertical padding and a top hairline

Class	Meaning
<code>.bay--steel</code>	Same, on the slightly lighter graphite background
<code>.shell</code>	The centred content column (max 1440px, side gutters)
<code>.titleblock</code>	Section header: mono tag + big heading + optional paragraph
<code>.split</code>	Two-column grid (60/40); <code>.split--flip</code> swaps the column order
<code>.note</code>	Small uppercase JetBrains Mono label — the "dimension callout" voice
<code>.ticked</code>	Adds the red corner registration marks to any box
<code>.frame</code>	An image with border and the little caption chip
<code>.reel</code>	A crossfading slideshow — put several <code></code> inside, first one with class <code>is-on</code>
<code>data-reveal</code>	Element fades/slides in when scrolled into view (see §10)
<code>.o</code>	Element takes part in the page-load stagger sequence

To add a brand-new section to any page: copy an existing `<section class="bay">...</section>` block, change the words, keep the classes.

6. The design system (`css/style.css`)

Everything visual comes from **CSS custom properties** ("tokens") declared once at the top of the file in `:root { ... }`. Change a token, and every use of it across the whole site updates.

The tokens that matter

```
:root {
  /* surfaces (darkest -> lightest) */
  --void:      #08090A; /* page background */
  --graphite:  #0F1112; /* panels, cards */
  --steel:     #16191B; /* hover states */

  /* lines */
  --edge:      #24292C; /* hairline borders everywhere */

  /* text */
  --chalk:     #F1EFEA; /* headings – warm off-white */
  --chalk-2:   #C2C0BA; /* body copy */
  --aluminum:  #838A8F; /* muted copy */

  /* the one accent hue, as a 3-stop ramp */
  --red:       #C8102E; /* borders, marks, fills */
  --red-ink:   #F2647A; /* red TEXT on dark (readable) */
  --red-fill:  #A50D26; /* button fills under white text*/

  /* type scale (fluid: min, preferred vw, max) */
  --t-mega:    clamp(2.75rem, 9vw, 7.5rem); /* hero display */
  --t-xxl:     clamp(2.4rem, 5.6vw, 4.75rem); /* page h1 */
  --t-xl:      clamp(2rem, 4.4vw, 3.5rem); /* section h2 */
  /* ... down to --t-note (11px mono labels) */

  /* spacing scale --s-1 (4px) ... --s-10 (128px) */
  /* motion: --ease-out, --dur-1/2/3 (180/450/900ms) */
}
```

Why three reds? One red cannot be both a readable *text* colour on a near-black page AND a *background* under white text — the contrast math pulls in opposite directions. So: `--red` is the brand mark, `--red-ink` is lightened for text, `--red-fill` is darkened for buttons. All three pass WCAG AA contrast in the positions they're used.

Where to change common things

Want to change	Edit
Team accent colour	The three <code>--red*</code> tokens (keep the light/dark relationship)
Fonts	The two <code><link></code> tags in each page head + <code>--display/--body/--mono</code> tokens
Page background	<code>--void</code> / <code>--graphite</code>
Base text size	<code>--t-base</code>
How big the hero type is	<code>--t-mega</code>
Section vertical spacing	<code>--bay</code>

The rest of the stylesheet is organised into labelled blocks — NAV, HERO, FRAME, SPECS, STAGE, CARDS, ROSTER, FORM, FOOTER, REVEAL SYSTEM — in the same order sections appear on the pages. Search for the block comment.

7. The JavaScript files

js/site.js — functional behaviour

Block	What it does
Nav toggle	Opens/closes the mobile burger menu, keeps <code>aria-expanded</code> correct
Image reels	Every <code>.reel</code> crossfades its <code></code> children every 4.2s; pauses when off-screen
Blog renderer	Fetches <code>data/posts.json</code> , sorts newest-first, builds the card grid into <code>#feed</code> (blog page) and the top-3 into <code>#latest</code> (home). Empty array -> renders the "Coming soon" panel
Post modal	Clicking a card opens the full post in an overlay; closes on x, backdrop click, or Escape
Contact form	Intercepts submit, POSTs the fields to Web3Forms as JSON, shows the inline success/error status line

js/motion.js — animation behaviour

Block	What it does
Page-load sequence	Adds <code>is-loaded</code> to <code><body></code> when the page is ready; elements with class <code>.o</code> then stagger in. A 1.6s timeout guarantees content never stays hidden
Reveal system	One <code>IntersectionObserver</code> watches every <code>[data-reveal]</code> element and adds <code>is-in</code> when it scrolls into view (see §10)
Counters	Elements with <code>data-count="25"</code> count up from 0 when first visible
Nav tuck	Header hides when you scroll down past 220px, returns when you scroll up; also drives the red progress hairline
Magnetic buttons	Elements with <code>data-magnetic</code> lean toward the cursor (pointer devices only)
GSAP layer	<code>data-drift</code> = subtle parallax on images; <code>data-wave</code> = children of a grid stagger in; <code>data-nudge</code> = horizontal rail gives a little "I scroll sideways" hint

Everything respects the user's *reduced motion* OS setting — with it on, all animation is skipped and content just appears.

js/rig.js — the 3D robot

Covered fully in §9. Summary: loads `models/kitbot.glb`, mounts it on the home hero (`<canvas id="rig">`, slow orbit + pointer parallax) and on the robot page (`<canvas id="exploded">`, scroll-driven exploded view).

8. TUTORIAL: Posting to the Logbook

The blog has **no admin panel and needs none**. All posts live in one file:

```
data/posts.json
```

The pages build themselves from it. Post = add an entry to this file, push.

8.1 The format

`posts.json` holds a JSON **array** (square brackets) of post **objects** (curly braces). Right now — pre-season — it is empty:

```
[]
```

Each post object has exactly these seven fields:

Field	Type	Rules
<code>id</code>	text	Unique slug, lowercase-with-hyphens. Never shown; used to tell posts apart
<code>title</code>	text	The headline on the card and in the reader
<code>date</code>	text	Must be YYYY-MM-DD (e.g. <code>2026-01-10</code>). Controls sort order — newest first
<code>author</code>	text	Shown next to the date
<code>image</code>	text	Path to the header photo, e.g. <code>images/blog/kickoff.jpg</code>
<code>excerpt</code>	text	1–2 sentence teaser shown on the card
<code>body</code>	text	The full post as HTML — paragraphs wrapped in <code><p>...</p></code>

8.2 Demo — your first real post, step by step

Step 1 — prepare the photo. Pick a good photo. Resize to about **1200×675** and compress it (drag onto tinypng.com; aim for under 300 KB). Name it lowercase with hyphens: `kickoff-2026.jpg`. Put it in `images/blog/` (create that folder if it doesn't exist yet).

(!) File names are case-sensitive on the live site. `Kickoff.JPG` and `kickoff.jpg` are different files there, even though Windows treats them as the same. Always lowercase.

Step 2 — edit `data/posts.json`. Open it in VS Code. Replace the empty `[]` with:

```
[
  {
    "id": "kickoff-2026",
    "title": "Kickoff – REBUILT revealed",
    "date": "2026-01-10",
    "author": "Ava R., Business Lead",
    "image": "images/blog/kickoff-2026.jpg",
    "excerpt": "Six weeks on the clock. The whole team watched the game reveal in Room B2, then argued strategy over pizza until six.",
    "body": "<p>FIRST dropped the 2026 game this morning and the shop was full by nine. First impressions: this is a cycling game, and intake speed is going to matter more than climb points.</p><p>By lunch the whiteboard had three robot concepts. By six we
```

```

    had killed two of them.</p><p>Prototyping starts Monday. We will post every week – hold
    us to it.</p>"
  }
]

```

Step 3 — check your commas. This is the #1 source of broken blogs:

```

[
  { ...post one... },      ← comma AFTER every post...
  { ...post two... },
  { ...post three... }    ← ...EXCEPT the last one
]

```

If you're not sure, paste the whole file into jsonlint.com — it will point at the exact problem line.

Step 4 — preview locally. Live Server -> open `blog.html`. Your post appears as a card; click it to read the full body in the overlay. The home page "Logbook" section shows it too. (The "Coming soon" panel disappears automatically the moment the array is non-empty.)

Step 5 — publish. GitHub Desktop -> you'll see `posts.json` and your new image in Changes -> write a summary like `Post: kickoff recap` -> **Commit to main** -> **Push origin**. Live in about a minute.

8.3 Formatting inside body

The `body` field is HTML, so you can use:

```

<p>A normal paragraph.</p>
<p>Some <strong>bold</strong> and <em>italic</em> text.</p>
<p>A <a href='https://www.thebluealliance.com/team/1635'>link</a>.</p>
<img src='images/blog/extra-photo.jpg' alt='The prototype on the bench'>
<ul><li>a bullet</li><li>another bullet</li></ul>

```

Two gotchas:

- Inside `body`, prefer **single quotes** for HTML attributes (`<a href='...'>`) so they don't collide with the JSON double quotes. If you must use a double quote in the text itself, escape it: `\"`.
- Keep it to one line. JSON strings cannot contain real line breaks — the `<p>` tags are your paragraphs.

8.4 Editing and deleting

- **Edit:** change the fields, save, push. Done.
- **Delete:** remove that post's whole `{ ... }` block (and the now-extra comma), save, push.
- **Ordering** is automatic by `date` — you never need to reorder the file.

9. TUTORIAL: The 3D robot, from zero

This section teaches the concepts from scratch, then maps them to our real code in `js/rig.js`, so you can modify it or build your own next time.

9.1 The four things every three.js program has

Think of shooting a film:

three.js object	Film-set equivalent
Scene	The stage — everything you place lives in it
Camera	The camera — where you look from, what's in frame
Renderer	The film crew — draws what the camera sees onto a <code><canvas></code>
Mesh	An actor — geometry (shape) + material (surface)

The smallest complete program — paste this into an empty HTML file and open it with Live Server; you'll see a spinning red box:

```
<canvas id="c" style="width:100%;height:400px"></canvas>
<script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>
<script>
  // 1 · the stage
  const scene = new THREE.Scene();

  // 2 · the camera: 40° lens, aspect ratio, near/far clip distances
  const camera = new THREE.PerspectiveCamera(40, 2, 0.1, 100);
  camera.position.set(0, 1, 5);      // step back and up a bit
  camera.lookAt(0, 0, 0);          // aim at the middle

  // 3 · the crew, drawing onto our canvas
  const canvas = document.getElementById('c');
  const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
  renderer.setSize(canvas.clientWidth, canvas.clientHeight, false);

  // 4 · an actor: box shape + red surface
  const box = new THREE.Mesh(
    new THREE.BoxGeometry(1, 1, 1),
    new THREE.MeshStandardMaterial({ color: 0xC8102E })
  );
  scene.add(box);

  // lights — MeshStandardMaterial is black without them
  scene.add(new THREE.AmbientLight(0xffffff, 0.6));
  const sun = new THREE.DirectionalLight(0xffffff, 1);
  sun.position.set(3, 5, 4);
  scene.add(sun);

  // the animation loop: ~60 times per second, move things, redraw
  (function loop() {
    requestAnimationFrame(loop);    // "call me again next frame"
    box.rotation.y += 0.01;         // spin a tiny bit each frame
    renderer.render(scene, camera);
  })();
</script>
```

Every 3D thing on our site is this exact pattern with more actors.

9.2 Loading a real model instead of a box

Robots aren't boxes. Real models ship as **glTF** files (**.gltf** = JSON, **.glb** = the same thing packed into one binary — smaller, what we use). three.js reads them with the **GLTFLoader** add-on:

```
<script src="https://cdn.jsdelivr.net/npm/three@0.128.0/examples/js/loaders/GLTFLoader.js"></script>
<script>
  new THREE.GLTFLoader().load('models/kitbot.glb', (gltf) => {
    scene.add(gltf.scene);          // drop the whole model into the stage
  });
</script>
```

Three problems always follow, and **rig.js** solves each:

1. **Wrong "up"**. CAD software uses Z-up; three.js uses Y-up, so the robot arrives lying on its back. Fix: rotate the whole model -90° around X: `model.rotation.x = -Math.PI / 2`.

2. **Wrong size/position**. The model might be metres wide or floating. Fix: measure it with `new THREE.Box3().setFromObject(model)`, then shift it so its centre is at the origin and scale it to a chosen size.

3. **You want to move PARTS, not just the whole thing**. Our **.glb** was deliberately built with named sub-assemblies (**bumpers**, **drive**, **launcher**...). `model.traverse(obj => ...)` visits every piece; we match pieces to groups by name and remember each one's starting position.

9.3 How the hero robot works ([index.html](#))

Open `js/rig.js` and find **MOUNT 1: hero**. In order:

- The `<canvas id="rig">` sits absolutely-positioned behind the hero text (`z-index: 0`; the text is `z-index: 2`).
- Two nested **Groups** do the placement:
`` `shift (WHERE on screen) -> spinner (ROTATION) -> robot` `` The order matters. Rotation inside, position outside = the robot spins in place at an offset. Swap them and it would orbit the screen centre like a moon.
- `frameHero()` computes the x-offset from the camera's field of view so the robot's centre lands at **80% of the viewport width** — that's why it never drifts into the headline at odd window sizes. It re-runs on resize.
- The loop adds: slow constant spin, a gentle bob (`Math.sin`), and pointer parallax — the mouse position eases into extra rotation (`px += (tx - px) * 0.04` is a smoothing filter: each frame move 4% of the remaining distance).
- Housekeeping that keeps laptops cool: pixel ratio capped at 2, an **IntersectionObserver** pauses rendering when the canvas is off-screen, and the OS "reduce motion" setting freezes the robot at a nice angle.

9.4 How the exploded view works (robot.html)

This is the scroll-driven "take it apart" section. Two halves: a CSS trick and a JS mapping.

The CSS trick — a pinned stage:

```
.stage      { height: 320svh; }      /* the section is 3.2 screens TALL */
.stage__pin { position: sticky; top: 0; height: 100svh; }
```

`.stage` gives you 2.2 screens of scrolling *runway*. `.stage__pin` (containing the canvas) is *sticky*, so while you scroll through the runway the canvas stays glued to the viewport. Result: the page scrolls, the picture holds still — the classic "scrollytelling" foundation. No library needed.

The JS mapping — scroll % -> explosion %:

```
function onScroll() {
  const r = stage.getBoundingClientRect(); // where is the stage?
  const span = r.height - window.innerHeight; // the runway length
  target = Math.min(1, Math.max(0, -r.top / span)); // 0 -> 1 progress
}
```

When the stage's top hits the viewport top, `-r.top` is 0 -> progress 0. When you've scrolled the whole runway, `-r.top` equals `span` -> progress 1. Clamped to [0, 1].

Using the number. Each sub-assembly has a hand-tuned "fly-away" vector:

```
var BLOW = {
  bumpers: [0, -0.55, 0], // drops down
  launcher: [0, 1.15, 0.25], // lifts up and back
  intake: [0, 0.05, 1.30], // slides out the front
  ...
};
```

And the explode function is one line of maths per part — *current = home + direction × progress*:

```
o.position.set(h.x + v[0]*e, h.y + v[1]*e, h.z + v[2]*e);
```

Two easing details make it feel physical rather than robotic:

- `e = t*t*(3-2*t)` — "smoothstep". Parts accelerate away and decelerate into place instead of moving linearly.

- `prog += (target - prog) * 0.08` — the displayed progress *chases* the scroll position with a lag, so even jerky scroll wheels produce silky motion.

The four text readouts and the little progress pips are driven by the same number: `index = floor(progress × 4)` picks which readout has the `is-in` class.

Recipe to build your own scroll-scrub from scratch:

1. Tall section (300–400svh) with a `position: sticky; top: 0; height: 100svh` child.
2. On scroll, compute progress 0->1 as above.
3. In a `requestAnimationFrame` loop, ease a display value toward that progress.
4. Drive *anything* with it — a 3D explosion, a video's `currentTime`, a counter, opacity of caption steps.

10. TUTORIAL: The scroll animations, from zero

All the small entrance animations use one pattern: **CSS defines the two states, JS only flips a class.**

10.1 The reveal system

CSS (simplified from `style.css`):

```
/* start state – gated behind .js so no-JS browsers see everything */
.js [data-reveal] {
  opacity: 0;
  transform: translateY(24px);
  transition: opacity .45s var(--ease-out), transform .45s var(--ease-out);
  transition-delay: calc(var(--i, 0) * 70ms); /* per-card stagger */
}
/* end state */
.js [data-reveal].is-in { opacity: 1; transform: none; }
```

JS (the whole engine, from `motion.js`):

```
const io = new IntersectionObserver((entries) => {
  entries.forEach((e) => {
    if (e.isIntersecting) { // element entered the viewport
      e.target.classList.add('is-in');
      io.unobserve(e.target); // animate once, then stop watching
    }
  });
}, { threshold: 0.14 }); // fire when 14% visible

document.querySelectorAll('[data-reveal]').forEach((el) => io.observe(el));
```

`IntersectionObserver` is a built-in browser API that tells you when an element scrolls into view — far cheaper than listening to every scroll event and measuring positions yourself.

To use it on anything new: add `data-reveal` to the element. Variants: `data-fx="clip"` (wipes downward), `data-fx="scale"`, `data-fx="left"`. For staggered grids give each child `style="--i:0"`, `--i:1`, `--i:2...`

10.2 The masked headline lines

The hero headline rises line-by-line out of invisible "slots":

```
<h1 data-hero-lines>
  <span class="line"><span>Six weeks.</span></span>
  <span class="line"><span>One crate.</span></span>
</h1>

.line { display: block; overflow: hidden; } /* the mask */
.js .line > span { display: block; transform: translateY(100%);
  transition: transform 1s var(--ease-out); }
.js .is-loaded .line > span { transform: none; }
.line:nth-child(2) > span { transition-delay: 90ms; } /* stagger */
```

Each inner span starts pushed below its own slot; `overflow: hidden` hides it; on load the parent gets `is-loaded` and the spans slide up into view.

10.3 The number counters

```
<span class="stat__n" data-count="25">0</span>
```

When first visible, JS runs a ~1.3s `requestAnimationFrame` loop that eases the displayed number from 0 to the target using $1 - (1-t)^3$ (starts fast, lands softly). The reveal observer triggers it, so it fires exactly once.

10.4 The GSAP layer

Three effects were nicer to write with GSAP's ScrollTrigger than by hand. They're all opt-in via data attributes, so you can apply them to new elements without touching JS:

Attribute	Effect	Example
<code>data-drift="-6"</code>	Image drifts up 6% of its height while it crosses the viewport (parallax)	hero-band photos
<code>data-wave</code> on a grid	The grid's children fade up one after another when the grid enters	spec board
<code>data-nudge</code> on a scroll rail	Rail auto-scrolls 90px and back once, hinting "I scroll sideways"	build-season rail

The pattern behind `data-drift`, for reference:

```
gsap.to(el, {
  yPercent: -6,
  ease: 'none',
  scrollTrigger: {
    trigger: el,
    start: 'top bottom',
    end: 'bottom top',
    scrub: 0.6
  },
  // scrubbed animations should be linear
  // when el's top hits viewport bottom
  // until el's bottom hits viewport top
  // tie progress to scroll, 0.6s lag
});
```

10.5 The safety rules the site follows

If you add animations, keep these — they're the difference between "polished" and "annoying":

1. Only animate `transform` and `opacity` (they're GPU-cheap; animating `width/top` causes layout jank).
2. Everything hidden-by-default must be gated behind the `.js` class.
3. Honour `prefers-reduced-motion` (our CSS block at the top of `style.css` already zeroes all animation globally when it's set).
4. One signature moment per page; everything else stays quiet.

11. The model build pipeline (`tools/build_kitbot.py`)

You only need this when the robot's CAD changes.

Input: `assets/kitbot/` — the Onshape per-part glTF export (621 files, 941 MB, gitignored). **Output:** `models/kitbot.glb` — one 3.5 MB binary with named sub-assemblies.

Run it with:

```
python tools/build_kitbot.py
```

What it does, in order:

1. **Scans** every `.gltf` and reads its mesh + colours.
 2. **Filters:** skips hardware (bolts/nuts/bearings — invisible at web scale), skips parts stuck at the origin (the export lost their placements), skips the "part-studio slab" (flat-layout duplicates), dedupes byte-identical stacked copies.
 3. **Groups** parts into named sub-assemblies (`bumpers`, `drive`, `launcher...`) — these names are what `rig.js` explodes.
 4. **Mirrors** symmetric parts whose twin's placement the export lost (side panels, side plates, guides).
 5. **Reconstructs** what the export dropped entirely, anchored to real geometry: the four roller stations, gold shafts, flywheel discs, carriage plates, flaps (hung through their measured mounting ring), churro standoffs, chassis rails + wheels, and the white "1635" bumper numerals.
 6. **Decimates** each group (vertex clustering) to a web-friendly triangle count and writes the single `.glb`.
- (!) The important caveat:** the per-part export keeps only ONE placement per part name, which is why step 5 exists. If the team ever re-exports from Onshape as a **single glTF of the whole assembly** (right-click the top-level assembly -> Export -> glTF), every instance keeps its true position and most of this script becomes unnecessary — the reconstruction guesswork can be deleted and the robot will match the CAD exactly.

12. The contact form

The form on `contact.html` really sends email, with no server of our own, via **Web3Forms** (free tier: 250 messages/month).

How it works:

1. The form holds a hidden `access_key` — our Web3Forms key. The key is *meant* to be public; it only works from our allowed domains (`noirhit.github.io`, `localhost`).
2. `site.js` intercepts the submit, packages the fields as JSON, and POSTs them to `https://api.web3forms.com/submit`.
3. Success -> green // **RECEIVED** status, form clears. Failure -> red status with a fallback email address.
4. A hidden `botcheck` checkbox is a honeypot: bots tick it, humans can't see it, Web3Forms discards ticked submissions.

Change who receives messages: log into web3forms.com with the team email and change the delivery address — no code edit needed. To CC more people, add inside the form:

```
<input type="hidden" name="cc" value="captain@example.com,mentor@example.com">
```

13. Publishing to the live site

The live site rebuilds automatically from the `main` branch.

Routine publish (GitHub Desktop):

1. Make and save your edits; check them with Live Server.
2. Open GitHub Desktop — the **Changes** tab lists every modified file.

Read the list. If something you didn't touch appears, find out why before committing.

3. Write a one-line summary (e.g. `Post: week 2 recap`) -> **Commit to main**.
4. **Push origin**.
5. ~60 seconds later, hard-refresh the live site (**Ctrl+Shift+R** — the normal refresh may show cached files).

Checking a deploy: repository -> **Actions** tab. Top row = latest run. Green check = live. Red X = failed -> click it and read which step failed.

Rules that keep deploys healthy:

- Compress images before adding them (tinyjpg.com). The deploy uploads the whole site every time; giant images make it slow and can time it out.
- Never commit `assets/kitbot/` (gitignore already blocks it).
- Lowercase file names, hyphens instead of spaces, always.

14. Cookbook — common jobs in 5 minutes

Post to the blog -> §8. `data/posts.json` + photo, push.

Update the roster names -> `about.html`, find § 04 — Roster, edit the `<li><b>Name</b><span class="note">Role</span></li>` lines. Add or remove lines freely — the grid reflows.

Change the roster count / stats -> `index.html`: the hero stat is `data-count="25"`; the "On the record" card says 25 students.

Swap the team photo -> drop the new photo in `images/` named `team-2026.jpg` (or update the `src` in `about.html`'s `.crew` block). 21:9-ish wide crops look best.

Add a sponsor -> `sponsors.html`, copy one `<article class="backer">...</article>` block, change the tier text, logo path (`images/sponsors/...`), name, line.

Change robot specs -> `robot.html`, the § 01 — By the numbers grid; each cell is a `.spec` div with label/value/small.

Add a whole page -> copy `blog.html` (simplest skeleton), rename, edit the `<title>`, the header/hero copy, and the `aria-current="page"` attribute in its own nav; then add a link to it in the `.nav__links` of all six other pages and the footer columns.

Update the 3D robot after CAD changes -> put the new export in `assets/kitbot/`, run `python tools/build_kitbot.py`, check locally, push (only `models/kitbot.glb` changes).

Season kickoff checklist -> post the first Logbook entry (the "Coming soon" panel disappears by itself), update the hero season tag in `index.html` (SEASON 2026 — REBUILT), update the robot name on `robot.html`.

15. Troubleshooting

Symptom	Cause -> fix
Blog shows "'// FEED UNREACHABLE" locally	You double-clicked the HTML file. Use Live Server (§3)
Blog broke after editing posts.json	Comma/quote error. Paste the file into jsonlint.com, fix the flagged line
New post/image works locally, broken live	Case mismatch — the live server is case-sensitive. Make the filename and the JSON path identical, all lowercase
Robot doesn't appear at all	Console (F12) will say why. Usually: <code>models/kitbot.glb</code> missing/renamed, or GLTFLoader <code><script></code> tag removed
Robot is a tiny 300×150 patch	The canvas lost its CSS size — <code>#rig/#exploded</code> must keep <code>width:100%;height:100%</code>
Everything invisible, no animations	A JS error stopped <code>motion.js</code> before it flagged the page. F12 -> Console -> fix the first red error. (Content only stays hidden if <code><html></code> has the <code>js</code> class and the reveal never fires)
Changes pushed but live site unchanged	Hard-refresh (Ctrl+Shift+R). Still stale -> repo Actions tab: is the deploy green?
Deploy fails with "Timeout reached"	Almost always oversized images. Compress anything over ~500 KB and push again
Form says <code>// TRANSMISSION FAILED</code>	Domain not on the Web3Forms allowlist, or their API hiccuped. Check the key's allowed domains at web3forms.com
Mobile menu won't open	The burger only renders below 880px width; if broken elsewhere, check Console for a <code>site.js</code> error

16. Glossary

Term	Meaning
Repository (repo)	The project folder as tracked by git, including its full history
Commit	A saved snapshot of your changes with a message
Push	Uploading your commits to GitHub (which triggers the deploy)
GitHub Pages	GitHub's free static-site hosting — serves our <code>main</code> branch
CDN	A public server that hosts common libraries (we load <code>three.js</code> and <code>GSAP</code> from CDNs)
JSON	A strict text format for data. Our blog database. Bracket/comma rules are unforgiving
<code>fetch()</code>	The browser function for loading a file/URL from JavaScript
<code>IntersectionObserver</code>	Browser API that reports when an element scrolls into view
<code>requestAnimationFrame</code>	"Run this function right before the next screen repaint" — the heartbeat of all smooth animation
WebGL	The browser's 3D drawing API; <code>three.js</code> is a friendly wrapper around it
glTF / GLB	The standard 3D model format for the web; GLB is its single-file binary form
Mesh	One 3D object: a geometry (shape) plus a material (surface)
Decimation	Reducing a model's triangle count while keeping its shape
Token (CSS custom property)	A named value like <code>--red</code> , defined once and reused everywhere
<code>clamp(min, preferred, max)</code>	CSS function behind our fluid type — scales with the viewport between hard limits
Sticky positioning	CSS <code>position: sticky</code> — element scrolls normally, then pins in place; the engine behind the exploded view
Honeypot	An invisible form field that only bots fill in — free spam filtering

Handbook v1 · written for the 2026 season · lives at `HANDBOOK.md` in the repo root. When the site changes, change this file in the same commit.